

PumpScript

AI Usage Journal — Part 1

CSE 341 · Concepts of Programming Languages

Gebze Technical University · 8 May 2026

Student Name	Yusuf Eren Nalbant
Student No	220104004049
AI Tool Used	Claude Sonnet 4.6 (claude.ai)
Entries	4 entries (Part 1)
Period	4 May 2026 — 8 May 2026

Journal Entries

Entry #1 of 4 — 4 May 2026, 21:00Phase: Design | Claude Sonnet 4.6

Goal: Choose a domain for PumpScript and decide what the language should be able to express.

Prompt:

I need to design a domain-specific language for my CSE 341 course project. The domain is sports. I am thinking about set count as int, weight as float, muscle group as string. Can you help me decide what the language should be able to do and what constructs it needs?

Response (summary): Claude suggested narrowing the domain from "sports" to "workout routines" specifically, and proposed four main constructs: an **exercise** record for individual movements, a **routine** declaration for grouping exercises, a **rest** statement for rest periods, and control structures (**repeat**, **if**) for dynamic plans. It also suggested adding a **bool** field **completed** to track whether an exercise was done.

✓ Accepted

Narrowing the domain to workout routines was the right call — it gave the language a clear identity. The four constructs (exercise, routine, rest, control structures) became the core of PumpScript. The **completed: bool** field was a good addition that I had not thought of.

× Rejected / Modified

Claude initially suggested passing **weight** as a routine parameter so all exercises share one value. I rejected this because in real training programs each exercise uses a different weight (bench press at 80kg, chest fly at 20kg). Placing **weight** inside each **exercise** block is more realistic and more reliable.

! Errors Caught

No factual errors. However, Claude's initial domain was too broad — it suggested a generic "sports tracking" language. I had to push back and narrow the scope to workout routines specifically.

*** Reflection**

Choosing a domain sounds easy but is actually the hardest design decision. "Sports" alone is not enough — you need to know exactly what a user of the language needs to express. Talking through use cases with an AI helped me realize that a fitness coach's core need is describing exercise sequences with weights, reps, and rest periods — not tracking scores or statistics.

Entry #2 of 4 — 5 May 2026, 20:30 Phase: Design | Claude Sonnet 4.6

Goal: Decide between static and dynamic scoping for PumpScript and justify the choice using Sebesta's terminology.

Prompt:

I am designing PumpScript, a DSL for workout routines. I need to choose between static scoping and dynamic scoping. Can you explain the difference and tell me which one fits better for my language? Also what would break if I used the wrong one?

Response (summary): Claude explained static scoping (names resolved by source text position) vs dynamic scoping (names resolved by call stack at runtime), citing Sebesta §5.5. It demonstrated with a code example that under dynamic scoping, a caller's `sets` parameter could silently override the callee's `sets`, which would be completely unexpected for a fitness coach writing PumpScript. It recommended static scoping for reliability and predictability.

✓ Accepted

The concrete example of caller interference under dynamic scoping was exactly what I needed. I could immediately see why static scoping is the right choice for PumpScript: a fitness coach should never have to worry about which routine called which when looking up a variable. I accepted static scoping and used this reasoning in D1 §4.6.

× Rejected / Modified

Nothing was rejected. The explanation matched Sebesta §5.5 accurately.

! Errors Caught

No errors detected. I verified the explanation against my lecture notes and Sebesta Ch. 5 — the distinction between static and dynamic scoping was correctly described.

★ Reflection

I already knew the definitions of static and dynamic scoping from lectures, but I had not thought about what dynamic scoping would concretely break in a workout DSL. The AI's example made the abstract concept tangible. This taught me that design decisions are easier to justify when you have a concrete failure scenario, not just a theoretical argument.

Entry #3 of 4 — 6 May 2026, 19:00 [E1 — Mandatory]Phase: Design | Claude Sonnet 4.6

Goal: Ask Claude to generate an EBNF grammar for PumpScript from a one-paragraph description, then compare it with my own grammar. (*Mandatory Experiment E1*)

Prompt:

PumpScript is a domain-specific language for workout routines. A program consists of routine declarations and statements. A routine has a name, parameters with types (int, float, string, bool), and a body block. A body contains exercise records (with fields like sets, reps, weight, group, completed), rest statements (rest N seconds), repeat loops (repeat N times), and if/else statements. Expressions include arithmetic, comparison, and equality operators. Generate a complete EBNF grammar for this language in Sebesta style using = instead of ::=.

Response (key excerpt):

```
<program> = { <routine_decl> | <statement> }
<routine_decl> = "routine" IDENT "(" [<param_list> "]" <block>
<param_list> = <param> { "," <param> }
<block> = "{" { <statement> } "}"
<statement> = <exercise_block> | <rest_stmt> | ...
<expr> = <expr> ("+" | "-") <term> | <term>
<term> = <term> ("*" | "/" ) <factor> | <factor>
```

✓ Accepted

The overall structure was correct: `program`, `routine_decl`, `param_list`, `block`, and `statement` all matched my own grammar closely. The high-level organization was a good starting point.

✗ Rejected / Modified

Critical error — right-recursive expression rules. Claude wrote: `<expr> = <expr> ("+" | "-") <term> | <term>` This is *right-recursive*, which produces right-associative parsing for + and -. My grammar uses the iterative { ... } construct to achieve left-associativity, which is the standard for arithmetic operators. I kept my own expression rules.

Missing operator precedence levels. Claude collapsed all arithmetic into two levels (`expr` and `term`) but missed the `comparison` level for relational operators (`>`, `<`, `>=`, `<=`). My grammar has five levels: `expr` → `comparison` → `term` → `factor` → `unary` → `primary`.

! Errors Caught

Two bugs detected: (1) Right-recursive expression rules produce right-associativity — incorrect for subtraction and division. Detected by mentally tracing $10 - 3 - 2$: Claude's grammar would parse it as $10 - (3 - 2) = 9$ instead of $(10 - 3) - 2 = 5$. (2) Missing **comparison** level means relational operators have the same precedence as arithmetic operators — incorrect.

★ Reflection

AI is effective at generating the scaffold of a grammar (rule names, overall structure) but unreliable on the details that encode a language designer's intent — specifically operator precedence and associativity. These are exactly the parts that require careful thought and cannot be delegated. Next time I will include an explicit precedence table in the prompt and always test associativity with a concrete example before accepting any expression grammar.

Entry #4 of 4 — 7 May 2026, 22:00 Phase: Implementation | Claude Sonnet 4.6

Goal: Generate the lexer and recursive-descent parser for PumpScript in Python, consistent with the EBNF grammar from D1.

Prompt:

Here is the complete EBNF grammar for PumpScript [pasted grammar]. Write a lexer in Python that tokenizes source code into token types: **KEYWORD**, **IDENT**, **INT_LIT**, **FLOAT_LIT**, **STRING_LIT**, **OP**, **SEP**, **EOF**. Then write a recursive-descent parser that builds an AST using node classes: **Program**, **RoutineDecl**, **Param**, **Block**, **ExerciseBlock**, **Field**, **RestStmt**, **RepeatStmt**, **IfStmt**, **RoutineCall**, **BinOp**, **UnaryOp**, **Literal**, **Ident**. Each node must have a **dump()** method for **-dump-ast** output. Include error messages with line numbers.

Response (summary): Claude generated a complete lexer with **advance()**, **peek()**, **current()** helpers, a **skip_whitespace_and_comments()** method, and separate read methods for each token type. The parser had one method per EBNF rule. The AST node classes each had a **dump()** method producing indented tree output.

✓ Accepted

The overall architecture was correct and clean: lexer and parser were properly separated, one parser method per EBNF rule, and the AST node hierarchy matched my design. The **expect()** helper method in the parser — which consumes a token and raises a **ParseError** with line number if it does not match — was a good pattern I kept. All three valid programs parsed successfully and all five error programs produced correct error messages.

× Rejected / Modified

The expression parser initially used right-recursion for **parse_term()** (same bug as in E1). I rewrote it as a loop using **while** to consume operators left-to-right, achieving left-associativity. I verified this by manually tracing $10 - 3 - 2$ through the corrected parser.

! Errors Caught

Same right-associativity bug as detected in E1 — confirming that AI consistently makes this mistake with expression grammars. Detected by running the test input `10 - 3 - 2` and checking the AST: the incorrect version produced `BinOp(-, 10, BinOp(-, 3, 2))` instead of the correct `BinOp(-, BinOp(-, 10, 3), 2)`.

★ Reflection

AI is excellent at generating parser boilerplate — the repetitive structure of recursive descent maps well to what language models do. But it makes the same associativity mistake every time, which suggests this is a systematic blind spot rather than a random error. Understanding *why* the bug happens (right-recursion encodes right-associativity) is more valuable than just fixing it — it means I can anticipate and catch it immediately in future sessions.